```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

EDA

```
df = pd.read_csv('/dataset_07_student_placement_eligit
```

```
df.shape
```

```
(1000, 7)
```

```
df.columns
```

```
Index(['cgpa', 'attendance_pct', 'coding_score', 'projects_completed',
       'internship_months', 'backlogs', 'target'],
      dtype='object')
```

```
df.head()
```

	cgpa	attendance_pct	coding_score	projects_completed	internship_months	ba
0	8.18	76.43	4	3	14	
1	9.28	46.50	3	3	50	
2	7.61	30.59	6	2	25	
3	7.47	50.98	0	4	56	
4	6.17	25.42	2	2	55	

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   cgpa                1000 non-null   float64
 1   attendance_pct      1000 non-null   float64
 2   coding_score        1000 non-null   int64  
 3   projects_completed  1000 non-null   int64  
 4   internship_months   1000 non-null   int64  
 5   backlogs            1000 non-null   int64  
 6   target              1000 non-null   int64  
dtypes: float64(2), int64(5)
memory usage: 54.8 KB
```

```
df.describe()
```

	cgpa	attendance_pct	coding_score	projects_completed	internship
count	1000.000000	1000.000000	1000.000000	1000.000000	1000
mean	7.262610	50.859510	2.999000	2.976000	30
std	1.255403	20.161735	1.711119	1.749424	17
min	4.000000	0.000000	0.000000	0.000000	1
25%	6.470000	37.447500	2.000000	2.000000	16
50%	7.270000	50.595000	3.000000	3.000000	30
75%	8.102500	64.027500	4.000000	4.000000	46
max	10.000000	100.000000	10.000000	9.000000	60

```
df.isnull().sum()
```

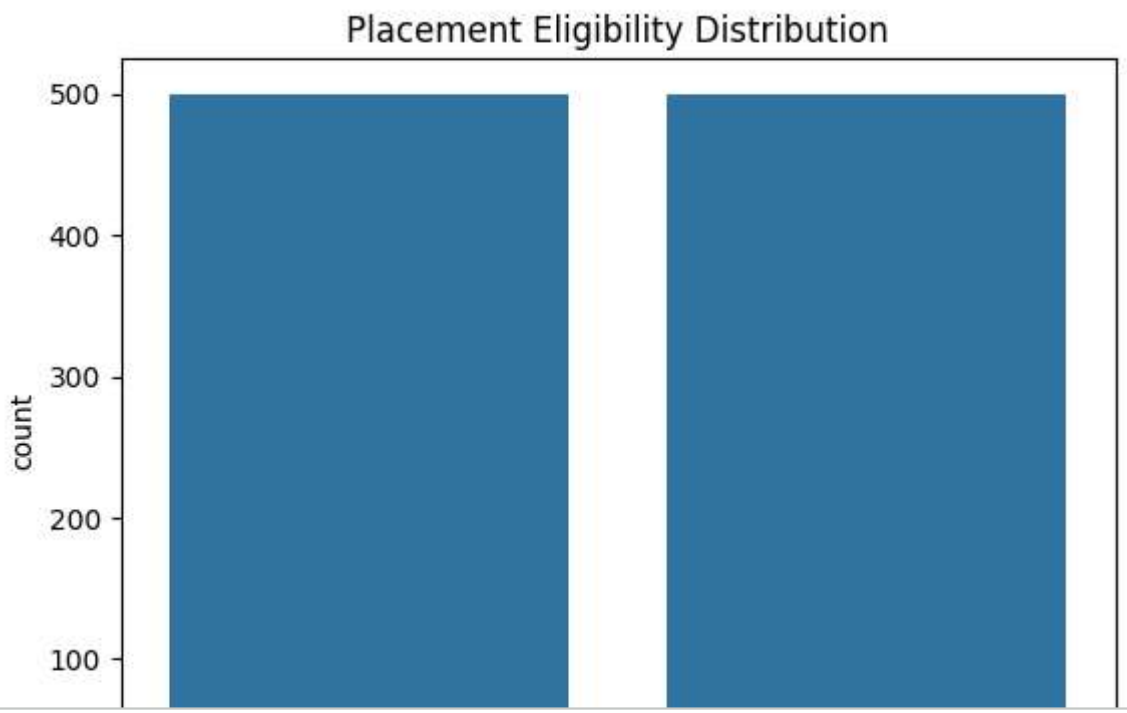
	0
cgpa	0
attendance_pct	0
coding_score	0
projects_completed	0
internship_months	0
backlogs	0
target	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

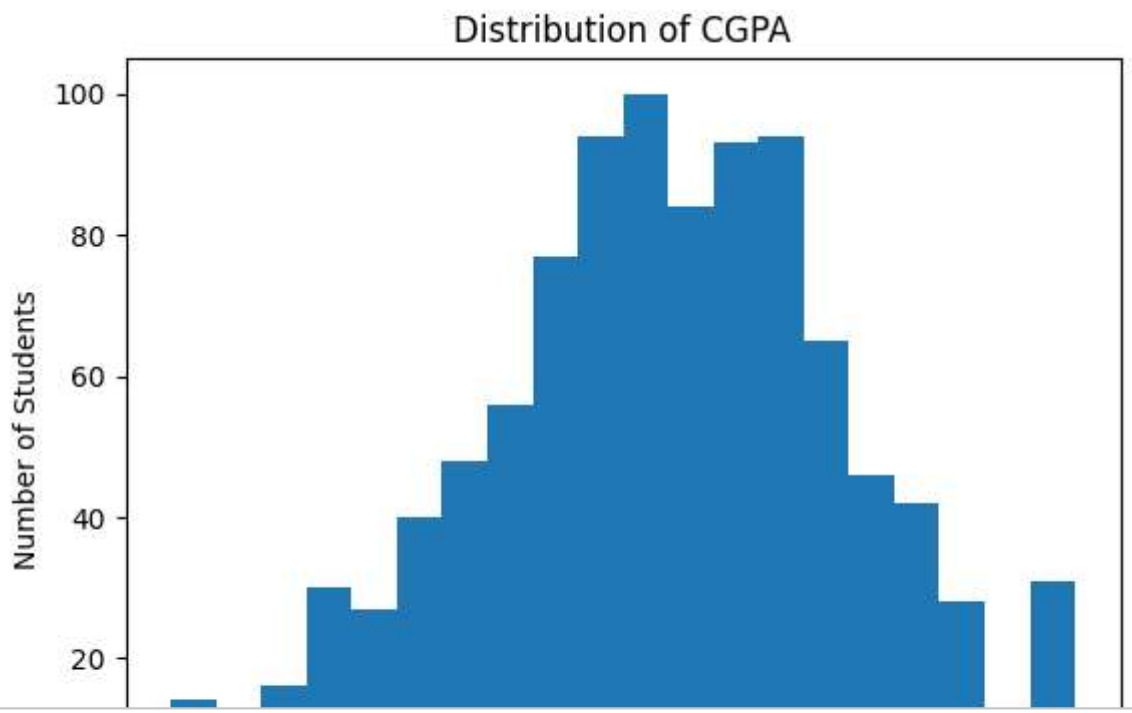
```
sns.countplot(x="target", data=df)
plt.title("Placement Eligibility Distribution")
plt.show()
```



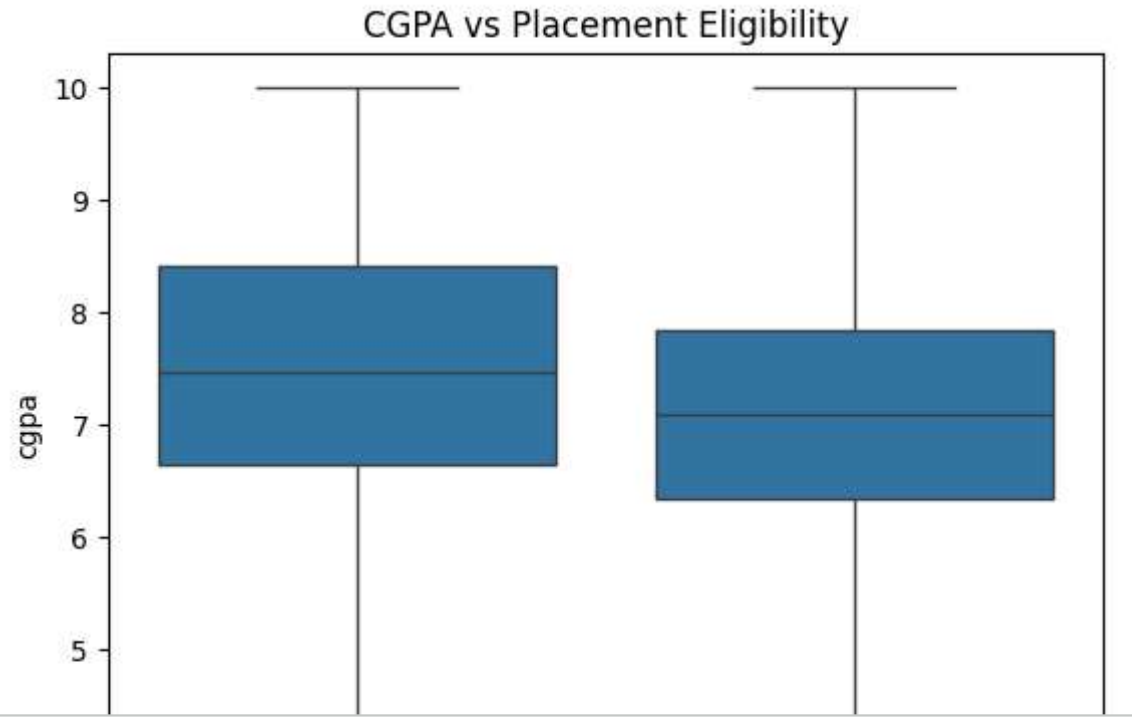

```
plt.hist(df["cgpa"], bins=20)

plt.xlabel("CGPA")
plt.ylabel("Number of Students")
```

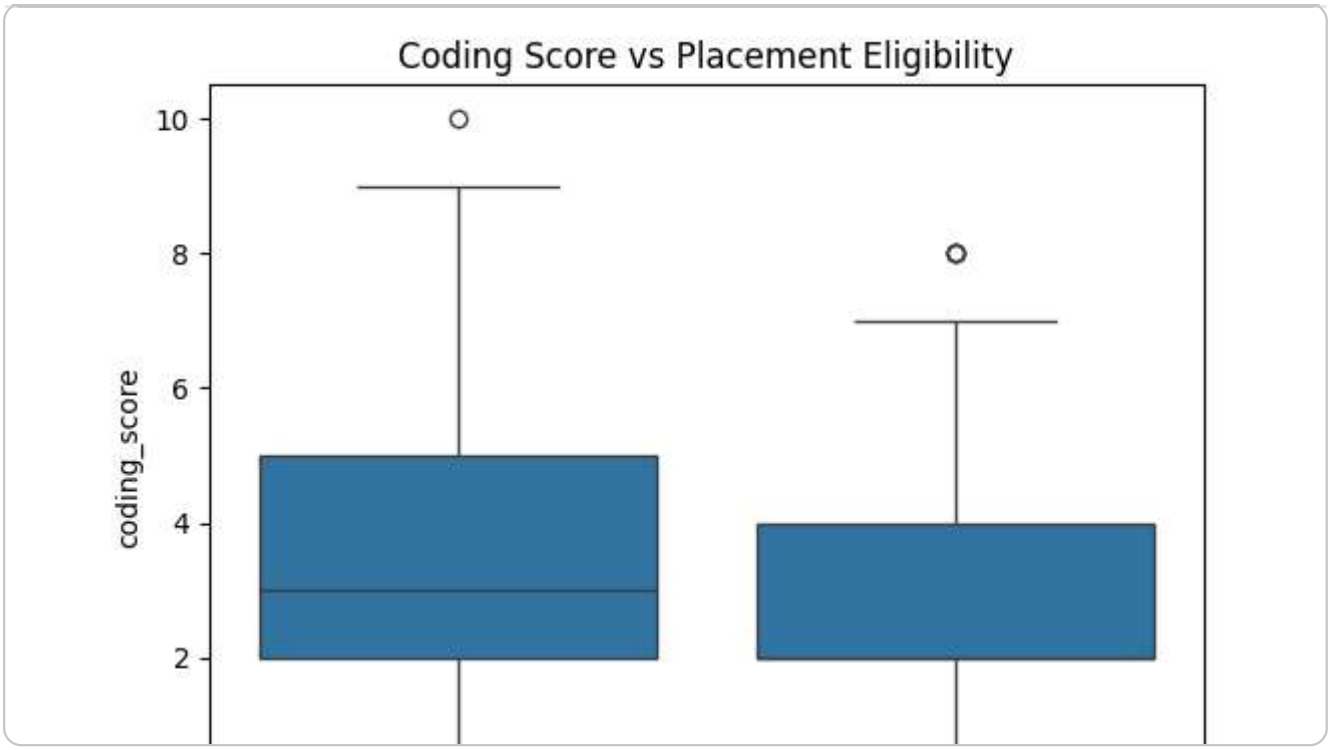
```
plt.title("Distribution of CGPA")
plt.show()
```



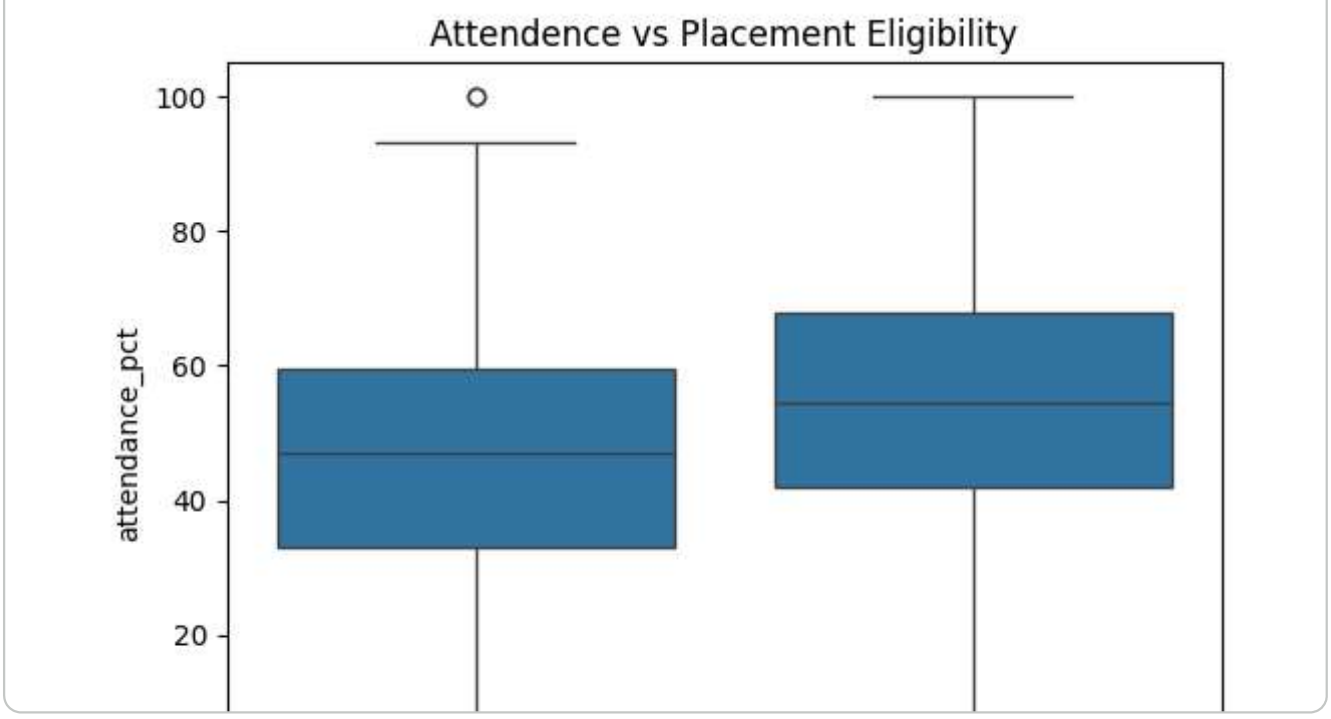

```
sns.boxplot(x="target",y="cgpa",data=df)
plt.title("CGPA vs Placement Eligibility")
plt.show()
```




```
sns.boxplot(x="target",y="coding_score", data=df)
plt.title("Coding Score vs Placement Eligibility")
plt.show()
```




```
sns.boxplot(x="target",y="attendance_pct", data=df)
plt.title("Attendence vs Placement Eligibility")
plt.show()
```

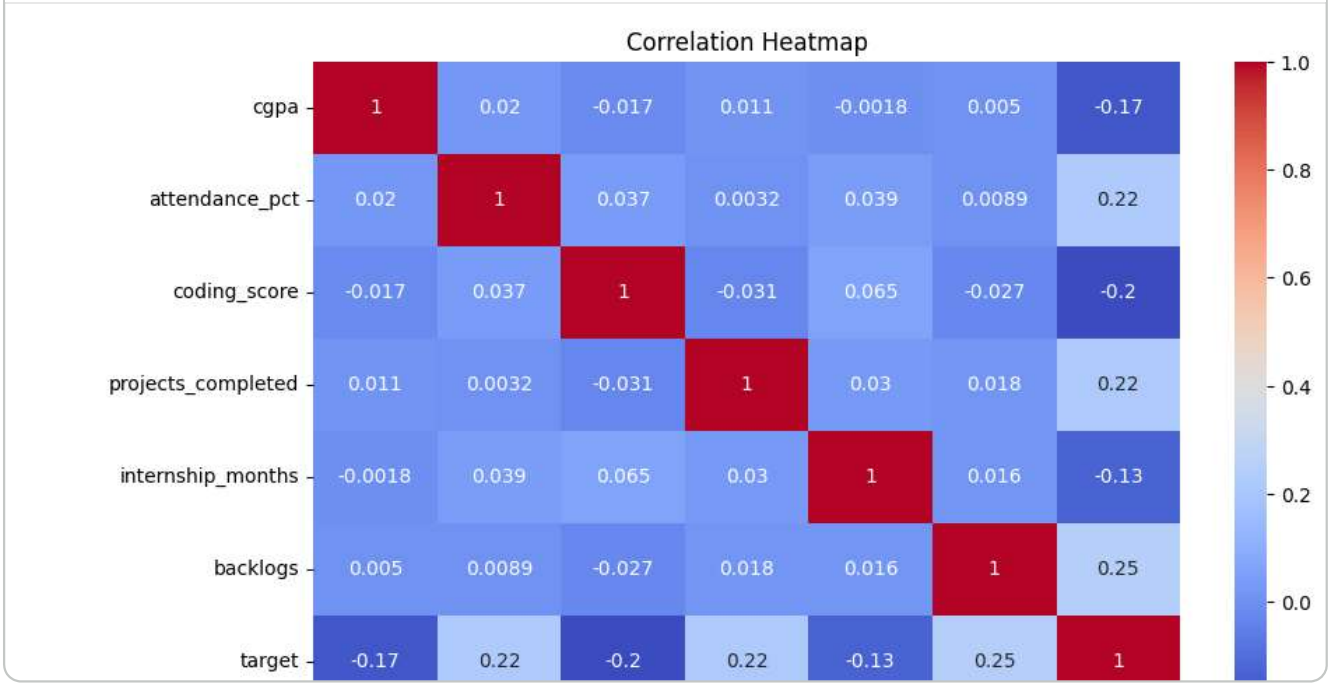


```
plt.figure(figsize=(10, 6))

sns.heatmap(df.corr(),annot=True,cmap="coolwarm")

plt.title("Correlation Heatmap")

plt.show()
```



Separate Feature & Target

```
X= df.drop("target",axis=1)
```

```
y = df["target"]
```

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_sta
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```

```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Logistic Regression Model

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train_scaled,y_train)
```

▾ LogisticRegression

```
LogisticRegression()
```

```
y_pred = model.predict(X_test_scaled)
```

```
y_pred[:10]
```

```
array([1, 0, 1, 1, 0, 1, 1, 0, 1, 0])
```

Accuracy

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test,y_pred)
```

```
accuracy*100,"%"
```

```
(69.0, '%')
```

Start coding or generate with AI.

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test,y_pred)
```

```
cm
```

```
array([[64, 27],
       [35, 74]])
```

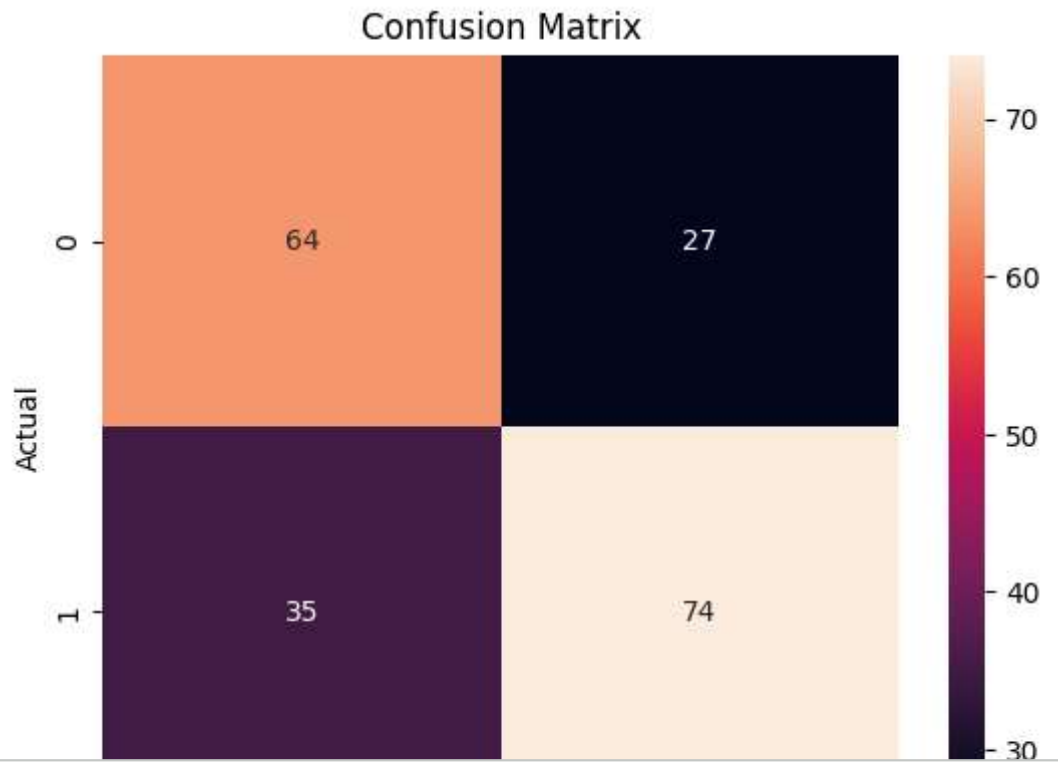
```
sns.heatmap(cm,annot=True,fmt="d")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```




```
from sklearn.metrics import classification_report
```

```
classification_report = classification_report(y_test,y_pred)
```

```
classification_report
```

```
'              precision    recall  f1-score   support\n\n           0       0.
65      0.70      0.67        91\n           1       0.73      0.68      0.70
109\n\n    accuracy                           0.69       200\n   macro avg
0.69      0.69      0.69       200\nweighted avg       0.69      0.69      0.69
200\n'
```